

Introdução ao Django REST Framework

Models e ORM

Aplicação de Gerenciamento de Pedidos com PostgreSQL e Django REST Framework

TADS — 4º Semestre | Prof. Plácido Neto

Objetivos da Aula

Ao final desta aula o aluno será capaz de:

- Compreender o papel do Django REST Framework na construção de APIs RESTful;
- Configurar um projeto Django integrado ao banco de dados PostgreSQL;
- Criar modelos de dados (Models) e mapeá-los para tabelas no banco de dados;
- Implementar Serializers para converter objetos Python em JSON (e vice-versa);
- Criar Views com ModelViewSet para expor operações CRUD pela API;
- Configurar rotas de URL usando o DefaultRouter do DRF;
- Documentar a API automaticamente com o Swagger (drf-yasg).

1. Contexto — A Aplicação de Gerenciamento de Pedidos

Nas aulas anteriores, aprendemos a criar e manipular tabelas diretamente com SQL. A partir desta aula, vamos dar um passo além: construir uma aplicação web completa que expõe esses dados por meio de uma API RESTful.

Para isso, usaremos o Django REST Framework (DRF), que nos permite criar APIs de forma organizada, segura e extensível, aproveitando todo o poder do ORM do Django para abstrair as operações no banco de dados.

1.1 Problema a ser resolvido

Imagine uma plataforma de e-commerce nos moldes da Amazon. Os usuários precisam conseguir:

- Cadastrar clientes, vendedores e itens;

- Visualizar os itens disponíveis;
- Adicionar itens ao carrinho e definir um endereço de entrega;
- Realizar um pedido e escolher a forma de pagamento;
- Consultar pedidos realizados, itens comprados e itens vendidos.

Toda essa lógica será exposta por meio de endpoints REST que retornam e recebem dados no formato JSON, permitindo que qualquer cliente — um aplicativo mobile, um front-end React/Angular ou um script Python — interaja com o sistema.

2. Configuração Inicial do Projeto

2.1 Criando o Ambiente Virtual e Instalando as Dependências

Antes de qualquer coisa, isole o projeto em um ambiente virtual. Isso garante que as versões das bibliotecas não conflitem com outros projetos Python na sua máquina.

Terminal — Ambiente Virtual

```
# 1. Criar e ativar o ambiente virtual
python3 -m venv env

# Linux / macOS
source env/bin/activate

# Windows
env\Scripts\activate

# 2. Instalar as dependências do projeto
pip install django                # Framework web principal
pip install djangorestframework   # Biblioteca para construção de APIs REST
pip install psycopg2-binary       # Driver de conexão com o PostgreSQL
pip install markdown              # Suporte a markdown na interface do DRF
pip install django-filter         # Filtros avançados nas consultas da API

# 3. Salvar as dependências em requirements.txt
pip freeze > requirements.txt
```

2.2 Criando o Projeto e o Aplicativo Django

Com o ambiente ativo, crie o projeto Django (estrutura principal) e um aplicativo interno responsável pela lógica de negócio:

Terminal — Criação do Projeto

```
# Criar o projeto principal chamado 'amazon'
django-admin startproject amazon

# Entrar na pasta do projeto
cd amazon

# Criar o aplicativo de backend
python manage.py startapp backend
```

2.3 Estrutura de Diretórios Resultante

Após os comandos acima, a estrutura do projeto será:

Estrutura de Diretórios

```
amazon/
```

```
├── manage.py           # Utilitário de linha de comando do Django
├── amazon/             # Módulo principal (configurações globais)
│   ├── __init__.py
│   ├── settings.py     # Configurações do projeto
│   ├── urls.py         # Roteamento global de URLs
│   ├── asgi.py
│   └── wsgi.py
├── backend/            # Aplicativo de negócio
│   ├── migrations/     # Histórico de migrações do banco de dados
│   ├── __init__.py
│   ├── admin.py        # Registro de modelos no painel de administração
│   ├── apps.py
│   ├── models.py       # Definição dos modelos (tabelas)
│   ├── serializers.py  # Conversão entre Python e JSON
│   ├── views.py        # Lógica de cada endpoint
│   ├── urls.py         # Roteamento local do aplicativo
│   └── tests.py
```

3. Configuração do PostgreSQL

3.1 Editando amazon/settings.py

O Django precisa saber a quais aplicativos pertence cada módulo e como se conectar ao banco de dados. Abra o arquivo `amazon/settings.py` e realize duas alterações:

Primeiro, registre os aplicativos recém-criados em `INSTALLED_APPS`:

settings.py — INSTALLED_APPS

```
# amazon/settings.py

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Aplicativos de terceiros
    'rest_framework',  # Django REST Framework
    'django_filters',  # Suporte a filtros avançados
    # Aplicativos do projeto
    'backend',         # Nossa lógica de negócio
]
```

Depois, configure a conexão com o PostgreSQL no dicionário `DATABASES`:

settings.py — DATABASES

```
# amazon/settings.py

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'amazon_db',          # Nome do banco de dados
        'USER': 'seu_usuario',       # Usuário do PostgreSQL
    }
}
```

```
'PASSWORD': 'sua_senha', # Senha do usuário
'HOST': 'localhost',      # Endereço do servidor
'PORT': '5432',           # Porta padrão do PostgreSQL
}
```

4. Modelos (Models)

No Django, cada classe que herda de `models.Model` representa uma tabela no banco de dados. Os atributos da classe correspondem às colunas dessa tabela. O ORM do Django é responsável por traduzir essas definições Python em instruções SQL.

4.1 Modelo Cliente

Vamos começar com um modelo simples para representar um cliente da plataforma:

backend/models.py — Modelo Cliente

```
# backend/models.py

from django.db import models

class Cliente(models.Model):

    nome = models.CharField(max_length=100)
    email = models.EmailField(unique=True) # UNIQUE no banco
    telefone = models.CharField(max_length=20, blank=True)
    data_cadastro = models.DateTimeField(auto_now_add=True) # Preenchido automaticamente

    class Meta:
        db_table = 'clientes' # Nome explícito da tabela no banco
        ordering = ['nome'] # Ordenação padrão nas consultas

    def __str__(self):
        # Retorna a representação legível do objeto
        return f'{self.nome} <{self.email}>'
```

4.2 Tipos de Campo Mais Usados

O Django disponibiliza uma variedade de tipos de campo para representar diferentes tipos de dados:

Campo	SQL Gerado	Quando usar
<code>CharField(max_length=N)</code>	<code>VARCHAR(N)</code>	Textos curtos: nomes, códigos
<code>TextField()</code>	<code>TEXT</code>	Textos longos: descrições, observações
<code>EmailField()</code>	<code>VARCHAR(254)</code>	E-mails (valida o formato automaticamente)
<code>IntegerField()</code>	<code>INTEGER</code>	Números inteiros
<code>DecimalField(max_digits, decimal_places)</code>	<code>NUMERIC(p,s)</code>	Valores monetários e precisão
<code>BooleanField()</code>	<code>BOOLEAN</code>	Flags verdadeiro / falso

DateField()	DATE	Datas sem horário
DateTimeField()	TIMESTAMP	Datas com horário
ForeignKey(Model, on_delete)	INTEGER (FK)	Chave estrangeira (relacionamento N:1)
ManyToManyField(Model)	Tabela pivô	Relacionamentos N:N

5. Serializers

Serializers são o coração da integração entre o banco de dados e a API. Eles realizam duas funções essenciais:

1. **Serialização:** converte um objeto Python (Model) em um dicionário Python, que por sua vez é transformado em JSON para envio ao cliente.
2. **Desserialização:** converte os dados JSON recebidos em uma requisição de volta para um objeto Python validado, pronto para ser salvo no banco.

5.1 Criando o Serializer do Cliente

Crie o arquivo backend/serializers.py e defina o serializer:

backend/serializers.py

```
# backend/serializers.py

from rest_framework import serializers
from .models import Cliente

class ClienteSerializer(serializers.ModelSerializer):

    class Meta:
        model = Cliente
        fields = '__all__' # Inclui todos os campos do modelo
        # Para expor apenas alguns campos, use uma lista:
        # fields = ['id', 'nome', 'email']
        # Para excluir campos, use:
        # exclude = ['data_cadastro']
```

5.2 Comportamento da Serialização

Com o serializer acima, um objeto Cliente é convertido para o seguinte JSON:

Exemplo de JSON gerado

```
{
  "id": 1,
  "nome": "Maria da Silva",
  "email": "maria@email.com",
  "telefone": "(84) 91234-5678",
  "data_cadastro": "2025-03-15T10:30:00Z"
```

```
}
```

6. Views

As Views definem o que acontece quando um endpoint da API é acessado. O Django REST Framework fornece o `ModelViewSet`, que implementa automaticamente as operações CRUD:

Ação	Método HTTP	URL	Descrição
list	GET	/clientes/	Lista todos os clientes
create	POST	/clientes/	Cadastra um novo cliente
retrieve	GET	/clientes/{id}/	Retorna um cliente pelo ID
update	PUT	/clientes/{id}/	Atualiza todos os dados
partial	PATCH	/clientes/{id}/	Atualiza campos específicos
destroy	DELETE	/clientes/{id}/	Remove o cliente

6.1 Implementando o ClienteViewSet

backend/views.py

```
# backend/views.py

from rest_framework import viewsets
from rest_framework.response import Response
from rest_framework import status
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import SearchFilter, OrderingFilter

from .models import Cliente
from .serializers import ClienteSerializer

class ClienteViewSet(viewsets.ModelViewSet):
    """
    ViewSet para o modelo Cliente.
    Fornece automaticamente os endpoints list, create, retrieve,
    update, partial_update e destroy.
    """
    queryset = Cliente.objects.all()
    serializer_class = ClienteSerializer

    # Habilita filtros, busca textual e ordenação via query params
    filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
    filterset_fields = ['nome', 'email'] # ?nome=Maria
    search_fields = ['nome', 'email'] # ?search=Maria
    ordering_fields = ['nome', 'data_cadastro'] # ?ordering=-data_cadastro
```

7. Configuração de URLs

O `DefaultRouter` do DRF gera automaticamente todas as rotas necessárias para um `ViewSet`, sem que precisemos declarar cada URL individualmente.

7.1 amazon/urls.py — Roteamento Global

amazon/urls.py

```
# amazon/urls.py

from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from backend import views

router = DefaultRouter()
router.register(r'clientes', views.ClienteViewSet, basename='cliente')

urlpatterns = [
    path('admin/', admin.site.urls),
    path('amazon_api/', include(router.urls)), # Todos os endpoints da API
]
```

7.2 Registrando Modelos no Admin

Para poder inspecionar e gerenciar os dados pelo painel de administração do Django, registre o modelo no arquivo `admin.py`:

backend/admin.py

```
# backend/admin.py

from django.contrib import admin
from .models import Cliente

@admin.register(Cliente)
class ClienteAdmin(admin.ModelAdmin):
    list_display = ('nome', 'email', 'telefone', 'data_cadastro')
    search_fields = ('nome', 'email')
    ordering = ('nome',)
```

8. Executando a Aplicação

8.1 Aplicando as Migrações

As migrações traduzem as definições dos Models para tabelas reais no banco de dados:

Terminal — Rodando o Projeto

```
# Gerar os arquivos de migração com base nos Models definidos
python manage.py makemigrations

# Aplicar as migrações ao banco de dados PostgreSQL
python manage.py migrate

# Criar um superusuário para acessar o painel de administração
python manage.py createsuperuser

# Iniciar o servidor de desenvolvimento
python manage.py runserver
```

8.2 Testando os Endpoints

Com o servidor rodando, acesse os seguintes endereços no navegador ou em um cliente REST como o Postman:

URL	O que faz
http://localhost:8000/admin/	Painel de administração do Django
http://localhost:8000/amazon_api/	Raiz da API — lista todos os endpoints
http://localhost:8000/amazon_api/clientes/	Lista todos os clientes (GET) / Cria novo (POST)
http://localhost:8000/amazon_api/clientes/1/	Detalha (GET), atualiza (PUT/PATCH) ou remove (DELETE) o cliente 1
http://localhost:8000/swagger/	Documentação interativa Swagger
http://localhost:8000/redoc/	Documentação ReDoc

8.3 Consumindo a API com Python (requests)

Além do navegador e do Postman, é possível consumir a API diretamente em um script Python. Isso é útil para testes de integração e para entender como um front-end interage com o back-end:

cliente_api.py — Consumindo a API com Python

```
# cliente_api.py
# Para instalar: pip install requests

import requests
import json
```

```
BASE_URL = 'http://localhost:8000/amazon_api/'

def listar_clientes():
    """Retorna todos os clientes cadastrados."""
    response = requests.get(BASE_URL + 'clientes/')
    if response.status_code == 200:
        return response.json()
    return []

def criar_cliente(nome, email, telefone=''):
    """Cadastra um novo cliente via POST."""
    payload = {'nome': nome, 'email': email, 'telefone': telefone}
    response = requests.post(BASE_URL + 'clientes/', json=payload)
    if response.status_code == 201:
        print(f'Cliente criado: {response.json()}')
    else:
        print(f'Erro {response.status_code}: {response.text}')

def buscar_cliente(cliente_id):
    """Busca um cliente específico pelo seu ID."""
    response = requests.get(BASE_URL + f'clientes/{cliente_id}/')
    if response.status_code == 200:
        return response.json()
    return None

if __name__ == '__main__':
    # Criar alguns clientes de teste
    criar_cliente('Maria da Silva', 'maria@email.com', '(84) 91111-2222')
    criar_cliente('João de Souza', 'joao@email.com', '(84) 93333-4444')

    # Listar todos os clientes
    print('\n--- Clientes cadastrados ---')
    for c in listar_clientes():
        print(f" [{c['id']}] {c['nome']} - {c['email']}")
```

9. Documentação Automática com Swagger (drf-yasg)

O Swagger é uma ferramenta open-source que gera documentação interativa para APIs REST a partir das definições de serializers e views do DRF. Com ele, qualquer desenvolvedor consegue explorar e testar os endpoints da API sem precisar ler o código-fonte.

9.1 Instalação e Configuração

Terminal

```
# Instalar o pacote
pip install drf-yasg
```

Adicione 'drf_yasg' ao INSTALLED_APPS em settings.py e configure as URLs:

amazon/urls.py — Swagger

```
# amazon/urls.py — adicionando as rotas do Swagger

from drf_yasg.views import get_schema_view
from drf_yasg import openapi
from rest_framework import permissions

schema_view = get_schema_view(
    openapi.Info(
        title='Amazon API',
        default_version='v1',
        description='API RESTful para gerenciamento de pedidos',
        contact=openapi.Contact(email='contato@amazon.com'),
        license=openapi.License(name='MIT License'),
    ),
    public=True,
    permission_classes=(permissions.AllowAny,),
)

urlpatterns += [
    path('swagger/', schema_view.with_ui('swagger', cache_timeout=0),
        name='swagger-ui'),
    path('redoc/', schema_view.with_ui('redoc', cache_timeout=0),
        name='redoc-ui'),
]
```

Acessando o Swagger

Com o servidor rodando, acesse <http://localhost:8000/swagger/> para visualizar a documentação interativa. Você pode testar todos os endpoints diretamente pelo browser, enviando requisições e inspecionando as respostas em tempo real.

10. Resumo da Aula

Nesta aula construímos, do zero, uma API RESTful funcional com Django REST Framework, cobrindo todo o fluxo de desenvolvimento:

Passo	Componente	Responsabilidade
1	Ambiente Virtual + pip	Isolar dependências do projeto
2	settings.py	Registrar apps e configurar PostgreSQL
3	models.py	Definir as tabelas (ORM)
4	makemigrations / migrate	Criar/atualizar tabelas no banco
5	serializers.py	Converter objetos ↔ JSON
6	views.py (ViewSet)	Implementar a lógica de cada endpoint
7	urls.py (Router)	Mapear URLs para ViewSets
8	Swagger (drf-yasg)	Documentar e testar a API interativamente

Referências

[Django REST Framework — Documentação Oficial](#)

[Django — Documentação Oficial](#)

[Repositório da Disciplina — lecture01-fundamentos](#)

[drf-yasg — Documentação Oficial](#)